MasterControl QAL Performance: Exploratory Data Analysis

Understanding the Mx Conversion Gap

Thomas Beck

2026-01-01

> **i** Project Context
>
> This notebook documents the discovery phase of the MasterControl Capstone project. The central question is why Mx leads convert to SQL at materially lower rates than Qx — and which lead characteristics, account attributes, and process signals most predictably separate successful leads from stalled ones.

Executive Summary

Mx vs. Qx Performance Snapshot

KPI	Mx	Qx	Gap
Conversion Rate (Lead to SQL+)	~12.6%	~19.7%	+7.1% pp
Decision-Maker Share (Director+)	~26%	~16%	+10% pp
Recycled Lead Rate	~78%	~70%	+8% pp

At $50 per sales call and ~$6,000 in pipeline value per SQL conversion, even a 2-point lift in Mx conversion rate translates to dozens of additional SQLs and meaningful revenue impact per quarter. Four findings define the Mx conversion problem:

1. **The gap is real.** Mx converts at roughly two-thirds the rate of Qx. The confidence bands don't overlap – this isn't noise.
2. **The problem is follow-through, not rejection.** Mx carries a substantially higher recycled rate than Qx. These leads weren't disqualified – they stalled. The gap is in the middle of the funnel, not at the gates.
3. **High-value segments exist.** Specific seniority-industry-model combinations convert at multiples of the Mx average. Site-scope contacts and premium-channel leads are underweighted targeting signals.

4. **The recycled pipeline is recoverable.** Over three thousand Mx leads sit in recycled status – not rejected, just stalled. Targeted re-engagement is the lowest-hanging fruit.

---

1. Introduction

MasterControl sells quality management and manufacturing execution software to regulated industries — primarily pharmaceutical, biotech, and medical device manufacturers. Two product lines are central to this analysis: **Mx** (manufacturing execution) and **Qx** (quality management systems).

Leads enter the pipeline as **Qualified Activity Leads (QALs)** — inbound contacts who have expressed interest through digital or event channels. Each QAL progresses through a defined funnel: Disqualified, Recycled, SQL (Sales Qualified Lead), SQO (Sales Qualified Opportunity), or Won. Conversion to SQL marks the handoff from marketing to active sales pursuit and is the primary measure of pipeline health.

Business problem: Mx converts QALs to SQL at a materially lower rate than Qx. The revenue impact is direct — a $50 cost per sales call and ~$6,000 value per SQL conversion means that even modest improvements in lead prioritization have measurable dollar consequences.

Analytic problem: Identify which lead characteristics — contact title, account industry and size, lead source channel, intent level, and temporal factors — are most predictive of conversion, with particular focus on diagnosing the Mx underperformance.

Target variable: `is_success` – equals 1 when a lead reaches SQL, SQO, or Won stage; 0 otherwise. The overall base rate is approximately 15%, meaning roughly 1 in 7 leads converts.

Purpose of this notebook: Document exploratory analysis of the QAL pipeline data, surface patterns across products, industries, contact seniority, and lead source channels, and produce a feature-enriched dataset suitable for downstream predictive modeling.

1.1 Guiding Questions

The following questions structure the exploratory analysis. Questions are listed upfront and addressed sequentially throughout the notebook.

1. What is the overall conversion rate, and how does it differ between Mx and Qx?
2. Where in the funnel do Mx leads exit at disproportionate rates?
3. Does lead cohort age at conversion differ between products, and what does that imply about the root cause?
4. Which contact title seniority levels are most associated with conversion?
5. Does the functional domain in a contact title (Quality, Regulatory, Mfg/Ops) predict conversion?
6. Does geographic or organizational scope ("Global" vs. "Site") in a title predict conversion?
7. Which specific title words and bigrams carry the strongest positive or negative conversion signal?

8. Which industry × seniority × manufacturing model combinations have the highest conversion rates?
9. Does lead source channel (Direct, SEO, Email, etc.) predict conversion likelihood?
10. Does lead priority or intent level predict conversion?
11. How does account tier (Small/Medium/Large) interact with industry to segment conversion potential?
12. How does conversion rate vary across sales territories?
13. What share of leads are missing key account or contact fields, and does missingness correlate with conversion?
14. Are recycled leads recoverable, and which segments hold the most dormant pipeline value?
15. Has the Mx conversion rate trended up or down across cohort quarters?

---

2. Data Overview

2.1 Schema & Variable Description

```
# load raw data to inspect schema
df_raw = pd.read_csv(RAW_DATA_PATH)
df_raw.columns = [c.strip().lower().replace(' ', '_').replace('/',
'_').replace('-', '_')
                  for c in df_raw.columns]

# build schema summary table
schema_rows = []
for col in df_raw.columns:
    dtype    = str(df_raw[col].dtype)
    n_unique = df_raw[col].nunique()
    n_null   = df_raw[col].isna().sum()
    sample   = df_raw[col].dropna().iloc[0] if df_raw[col].notna().any() else
"_"
    schema_rows.append({
        "Field": col,
        "Type": dtype,
        "Unique Values": n_unique,
        "Null Count": n_null,
        "Example": str(sample)[:40]
    })

schema_df = pd.DataFrame(schema_rows)
print_table(schema_df, caption=f"Dataset Schema — {len(df_raw):,} records ×
{len(df_raw.columns)} fields")
```

Dataset Schema — 16,815 records × 14 fields

Field	Type	Unique Values	Null Count	Example
acct_primary_site_function	str	78	6877	Food and beverage
acct_manufacturing_model	str	19	6875	Consumer Packaged Goods
acct_target_industry	str	5	45	Non-Life Science
contact_lead_title	str	5789	6410	QA Manager
qal_id	str	16815	0	a0dPQ000005WbfuYAC
contact_lead_id	str	15516	3	0030c00002XPeNGAA1
next_stage__c	str	8	483	SQL
priority	str	11	0	P1 - Webinar Demo
acct_territory_rollup	str	6	133	Americas
acct_tier_rollup	str	4	45	Medium
solution	str	14	0	Mx
solution_rollup	str	3	0	Mx
last_tactic_campaign_channel	str	12	0	Online Ads
qal_cohort_date	str	598	0	2025-06-16

The dataset contains QAL records exported from MasterControl's CRM. Each row represents one qualified inbound lead contact. Variable groups break down as follows:

- **Lead outcome:** `next_stage__c` — the pipeline stage reached (Disqualified, Recycled, SQL, SQO, Won). The basis for the binary target variable.
- **Product line:** `solution_rollup` — Mx (manufacturing execution) or Qx (quality management), with a small residual "Other" category.
- **Account attributes:** `acct_target_industry`, `acct_manufacturing_model`, `acct_primary_site_function`, `acct_territory_rollup`, `acct_tier_rollup` — describe the company associated with the lead. These fields have meaningful rates of missing or ambiguous values (see Section 2.2).
- **Contact attributes:** `contact_lead_title` — free-text job title of the inbound contact. Requires parsing to extract seniority, functional domain, and organizational scope.
- **Intent & source:** `priority` (P1 Website Pricing, P1 Contact Us, etc.) and `last_tactic_campaign_channel` (Direct/Inbound, SEO, Email, etc.) — describe how and with what expressed intent the lead arrived.
- **Temporal:** `qal_cohort_date` — the date the lead entered QAL status. Enables cohort-based trend analysis.

2.2 Missing Data

```
# key fields to audit
key_fields = [
    'acct_manufacturing_model', 'acct_primary_site_function',
    'acct_target_industry',     'acct_territory_rollup',
    'acct_tier_rollup',         'contact_lead_title',
    'priority',                 'last_tactic_campaign_channel'
]

missing_summary = []
for col in key_fields:
    if col not in df_raw.columns:
        continue
    n_na = df_raw[col].isna().sum()
    # count sentinel values as missing
    n_unknown = (df_raw[col].astype(str).str.strip().str.lower()
                 .isin(['unknown', 'not enough info', 'nan', ''])).sum()
    n_total = n_na + n_unknown
    missing_summary.append({
        'Field'                    : col,
        'True NA'                  : n_na,
        '"Unknown" / Sentinel'     : n_unknown,
        'Total Missing / Ambiguous': n_total,
        'Pct'                      : n_total / len(df_raw)
    })

missing_df = pd.DataFrame(missing_summary).sort_values('Pct', ascending=True)

# plot missing data
fig, ax = plt.subplots(figsize=(10, 4))

bar_colors = [COL_RISK if p > 0.3 else COL_GOLD if p > 0.1 else COL_NEUTRAL
              for p in missing_df['Pct']]
ax.barh(missing_df['Field'], missing_df['Pct'], color=bar_colors,
edgecolor='white')

for i, (_, row) in enumerate(missing_df.iterrows()):
    ax.text(row['Pct'] + 0.005, i,
            f"{row['Pct']:.1%}  (n={row['Total Missing / Ambiguous']:,})",
            va='center', fontsize=9)

ax.set_xlabel('Proportion Missing or Ambiguous', fontsize=11)
ax.set_title('Missing Data Audit — Key CRM Fields', fontweight='bold',
fontsize=14)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax.set_xlim(0, 1.05)
sns.despine()
plt.tight_layout()
```

```
plt.show()

# summary table
tbl = missing_df[['Field','True NA','"Unknown" / Sentinel','Total Missing /
Ambiguous','Pct']].copy()
tbl['Pct'] = tbl['Pct'].apply(lambda x: f"{x:.1%}")
print_table(tbl, caption="Missing Data Summary")
```

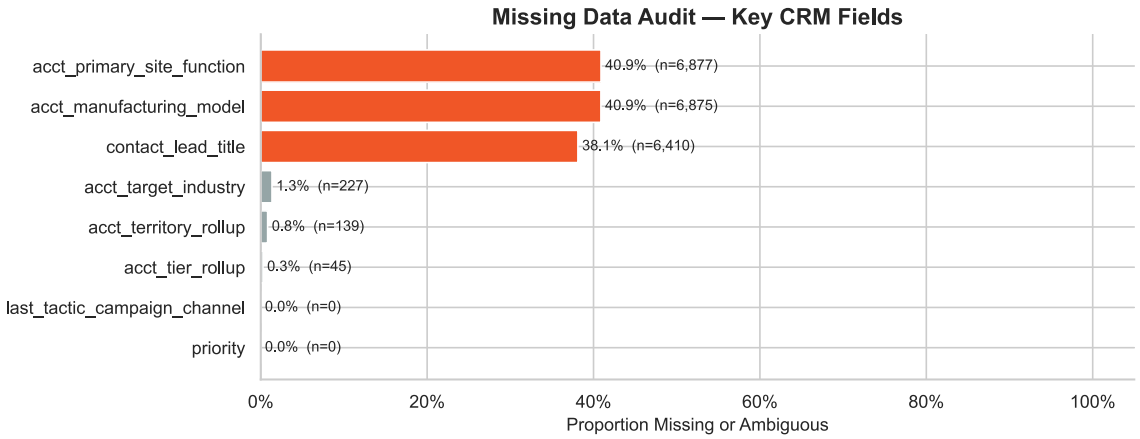

Figure 1: Scope of missing and ambiguous values across key CRM fields. Orange bars exceed 30% missing; gold bars exceed 10%.

Missing Data Summary

Field	True NA	"Unknown" / Sentinel	Total Missing / Ambiguous	Pct
priority	0	0	0	0.0%
last_tactic_campaign_channel	0	0	0	0.0%
acct_tier_rollup	45	0	45	0.3%
acct_territory_rollup	133	6	139	0.8%
acct_target_industry	45	182	227	1.3%
contact_lead_title	6410	0	6410	38.1%
acct_manufacturing_model	6875	0	6875	40.9%
acct_primary_site_function	6877	0	6877	40.9%

Account-level fields — particularly `acct_manufacturing_model` and `acct_primary_site_function` — carry the highest rates of ambiguity. These fields are populated by MasterControl's enrichment process, which uses AI-assisted web scraping to classify accounts; companies with minimal web presence are labeled "Not Enough Info." Critically, this label is not

random noise — it is itself an informative signal. Accounts that are hard to classify online tend to be smaller or less digitally mature, which affects conversion probability.

Proposed handling strategy:

- True NA values in categorical fields are imputed with an "Unknown" sentinel category preserved as a model input rather than dropped — the absence of information is predictive.
- Fields like `acct_manufacturing_model = "Not Enough Info"` are retained as a distinct category rather than coerced to missing. Analysis later in this notebook examines whether this category has a distinctive conversion profile (the "hidden gem" hypothesis).
- The `contact_lead_title` field is near-complete; the small number of blank titles are flagged in the `record_completeness` score engineered in Section 3.

---

3. Data Preparation & Feature Engineering

The raw CRM data requires substantial feature engineering before patterns become legible. The pipeline below standardizes column names, defines the binary target, and constructs five domain-informed feature families from the raw fields.

```
# segment subsets for repeated use
df_mx = df[df['product_segment'] == 'Mx']
df_qx = df[df['product_segment'] == 'Qx']

overview = pd.DataFrame({
    'Metric': [
        'Total Records', 'Mx Leads', 'Qx Leads',
        'Overall Conversion Rate', 'Mx Conversion Rate', 'Qx Conversion Rate',
        'Decision Makers (Director+)', 'Recycled (Near-Miss) Leads'
    ],
    'Value': [
        f"{len(df):,}",
        f"{len(df_mx):,}",
        f"{len(df_qx):,}",
        f"{df['is_success'].mean():.1%}",
        f"{df_mx['is_success'].mean():.1%}",
        f"{df_qx['is_success'].mean():.1%}",
        f"{df['is_decision_maker'].sum():,}
({df['is_decision_maker'].mean():.1%})",
        f"{len(df[df['outcome_tier']=='Near-Miss']):,}"
    ]
})
print_table(overview, caption="Dataset Overview Post Feature Engineering")
```

Dataset Overview Post Feature Engineering

Metric	Value
Total Records	16,815
Mx Leads	4,239
Qx Leads	12,547
Overall Conversion Rate	17.9%
Mx Conversion Rate	12.6%
Qx Conversion Rate	19.7%
Decision Makers (Director+)	3,066 (18.2%)
Recycled (Near-Miss) Leads	12,067

Five feature families are engineered from the raw CRM fields:

- **Title seniority** – parsed from free-text job titles (C-Suite, SVP, VP, Director, Manager, IC); the binary `is_decision_maker` flag captures Director and above.
- **Functional domain** – classifies titles into Quality, Regulatory, Mfg/Ops, IT, R&D, and PMO.
- **Organizational scope** – identifies whether a contact has global, regional, site-level, or standard authority.
- **Record completeness** – scores each lead on how fully account fields are populated, a proxy for how well-understood the account is in the CRM.
- **Temporal features** – cohort year/quarter and lead age in days (relative to the most recent cohort in the dataset), enabling trend and cohort maturity analysis.

---

4. The Conversion Gap

This section establishes the Mx vs. Qx performance gap and maps where leads exit the pipeline.

4.1 Mx vs. Qx Conversion Rate

Does Mx really convert worse, or is the gap just noise in the data? This is the first question to settle before digging into root causes.

```
df_main = df[df['product_segment'].isin(['Mx', 'Qx'])]

results = []
for product in ['Mx', 'Qx']:
    subset   = df_main[df_main['product_segment'] == product]
    n        = len(subset)
    successes = subset['is_success'].sum()
    rate, ci_low, ci_high = smoothed_conversion_rate(successes, n)
    results.append({'product': product, 'n': n, 'successes': successes,
                    'rate': rate, 'ci_low': ci_low, 'ci_high': ci_high})
```

```
results_df = pd.DataFrame(results)
overall_avg = df_main['is_success'].mean()
gap = results_df[results_df['product']=='Qx']['rate'].values[0] -
results_df[results_df['product']=='Mx']['rate'].values[0]

# bullet-style comparison chart
fig, ax = plt.subplots(figsize=(10, 3.5))

for i, row in results_df.iterrows():
    color = COL_SUCCESS if row['product'] == 'Mx' else COL_RISK
    # main bar
    ax.barh(row['product'], row['rate'], height=0.5, color=color, alpha=0.85,
            edgecolor='white', linewidth=1.5, zorder=3)
    # CI whiskers
    ax.plot([row['ci_low'], row['ci_high']], [i, i],
            color='black', linewidth=2.5, zorder=4, solid_capstyle='round')
    ax.plot([row['ci_low'], row['ci_high']], [i, i],
            '|', color='black', markersize=10, zorder=4)
    # rate label
    ax.text(row['rate'] + 0.008, i, f"{row['rate']:.1%}  (n={row['n']:,})",
            va='center', fontsize=12, fontweight='bold')

# overall avg reference line
ax.axvline(x=overall_avg, color=COL_NEUTRAL, linestyle='--', linewidth=1.5,
           label=f'Overall Avg: {overall_avg:.1%}', zorder=2)

# gap annotation
ax.annotate('', xy=(results_df.iloc[1]['rate'], 0.5),
            xytext=(results_df.iloc[0]['rate'], 0.5),
            arrowprops=dict(arrowstyle='<->', color=COL_ACCENT, lw=2))
ax.text((results_df.iloc[0]['rate'] + results_df.iloc[1]['rate']) / 2, 0.72,
        f'{gap:.1%} gap', ha='center', fontsize=11, fontweight='bold',
color=COL_ACCENT)

ax.set_xlabel('Conversion Rate (Lead to SQL+)', fontsize=11)
ax.set_title('The Mx Conversion Gap Is Real', fontweight='bold', fontsize=14)
ax.set_xlim(0, 0.30)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax.legend(loc='lower right', fontsize=9)
sns.despine()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "eda_conversion_gap.png", dpi=300)
plt.show()
```

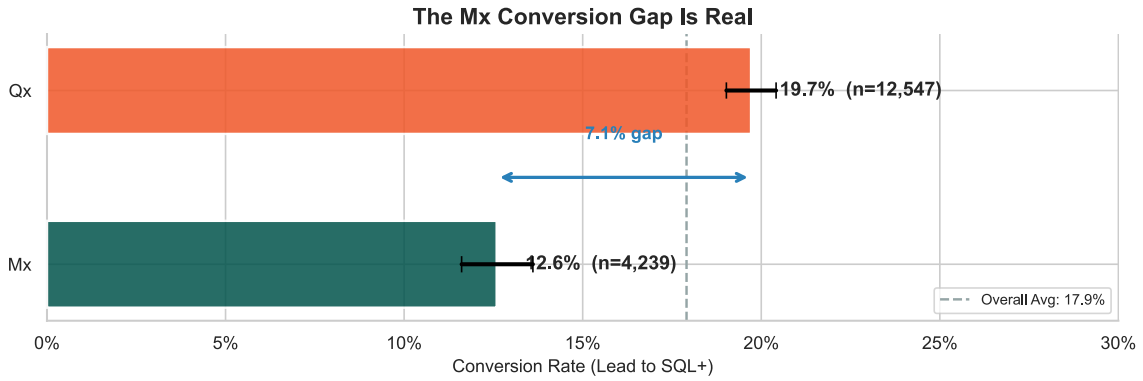

Figure 2: Qx outperforms Mx by approximately 7 percentage points. Confidence bands confirm the gap is real, not a sampling artifact.

The gap holds up under statistical testing – it's real. Mx converts at roughly two-thirds the rate of Qx, and the confidence bands don't overlap. Both products serve the same regulated-industry customers, so a 7-point gap this persistent points to something structural, not a campaign tweak away from closing.

4.2 Funnel Stage Distribution

The question is simple: are Mx leads getting rejected (bad targeting) or getting stuck (bad process)? A back-to-back comparison of where each product's leads end up answers this directly.

```
stage_order  = ['Disqualified', 'Recycled', 'SQL', 'SQO', 'Won']
df_products  = df[df['product_segment'].isin(['Mx', 'Qx'])].copy()
stage_counts = df_products.groupby(['product_segment',
'next_stage__c']).size().unstack(fill_value=0)
existing     = [s for s in stage_order if s in stage_counts.columns]
stage_counts = stage_counts[existing]
stage_pct    = stage_counts.div(stage_counts.sum(axis=1), axis=0) * 100

mx_pct = stage_pct.loc['Mx'] if 'Mx' in stage_pct.index else
pd.Series([0]*len(existing))
qx_pct = stage_pct.loc['Qx'] if 'Qx' in stage_pct.index else
pd.Series([0]*len(existing))

# butterfly (back-to-back) horizontal bar chart
fig, ax = plt.subplots(figsize=(10, 4.5))
y = np.arange(len(existing))

# Mx extends left, Qx extends right
ax.barh(y, -mx_pct.values, height=0.6, color=COL_SUCCESS, alpha=0.85,
        label='Mx', edgecolor='white', linewidth=1)
ax.barh(y, qx_pct.values, height=0.6, color=COL_RISK, alpha=0.85,
        label='Qx', edgecolor='white', linewidth=1)
```

```
# labels on each bar
for i, stage in enumerate(existing):
    mx_val = mx_pct[stage] if stage in mx_pct.index else 0
    qx_val = qx_pct[stage] if stage in qx_pct.index else 0
    ax.text(-mx_val - 0.8, i, f'{mx_val:.1f}%', va='center', ha='right',
fontsize=9,
            fontweight='bold', color=COL_SUCCESS)
    ax.text(qx_val + 0.8, i, f'{qx_val:.1f}%', va='center', ha='left',
fontsize=9,
            fontweight='bold', color=COL_RISK)

# highlight the recycled row
recycled_idx = existing.index('Recycled') if 'Recycled' in existing else None
if recycled_idx is not None:
    ax.axhspan(recycled_idx - 0.4, recycled_idx + 0.4, color=COL_GOLD,
alpha=0.12, zorder=0)
    recycled_mx = mx_pct['Recycled'] if 'Recycled' in mx_pct.index else 0
    recycled_qx = qx_pct['Recycled'] if 'Recycled' in qx_pct.index else 0
    ax.annotate(f'+{recycled_mx - recycled_qx:.1f} pp surplus',
                xy=(0, recycled_idx + 0.38), fontsize=9, fontstyle='italic',
                color=COL_ACCENT, ha='center', va='bottom')

ax.axvline(0, color='black', linewidth=1, zorder=3)
ax.set_yticks(y)
ax.set_yticklabels(existing, fontsize=11)
ax.set_xlabel('% of Product Leads', fontsize=11)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{abs(x):.0f}%'))
ax.set_title('Where Do Leads End Up? Mx (left) vs. Qx (right)',
             fontweight='bold', fontsize=14)
ax.legend(loc='lower right', fontsize=9)
sns.despine()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "eda_funnel_dropoff.png", dpi=300)
plt.show()
```

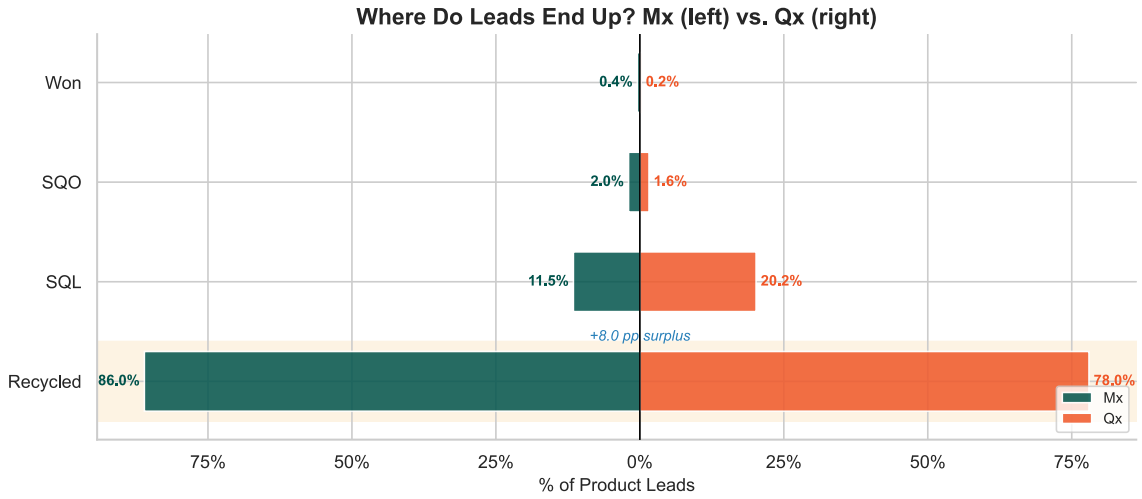

Figure 3: Butterfly chart showing where Mx and Qx leads land in the pipeline. The recycled stage is where Mx diverges.

Funnel Stage Distribution by Product

Stage	Mx (%)	Qx (%)
Recycled	86	78
SQL	11.5	20.2
SQO	2	1.6
Won	0.4	0.2

The butterfly chart makes the diagnosis clear. **The divergence is concentrated in the Recycled bucket**, where Mx carries a materially higher share than Qx. No leads in either product are formally disqualified – the data contains no Disqualified stage outcomes at all. Instead, leads that don't convert land in Recycled, and Mx accumulates more of them. These recycled leads aren't bad leads. They passed initial qualification – a human decided they were worth pursuing – but then stalled before reaching SQL. The problem is follow-through, not targeting. This points to sales process friction (cadence, enablement, messaging) as the primary lever.

---

5. Lead & Account Characteristics

This section examines how structural lead attributes – intent level, lead source channel, account tier, industry, and territory – relate to conversion. These are the dimensions that determine which types of inbound leads are worth prioritizing.

5.1 Lead Priority & Intent Level

Not all inbound actions signal the same buying intent. A lead who seeks out pricing has already self-selected into a buying mindset; a webinar registrant may just be browsing. The `priority` field captures this distinction directly.

```
if 'priority' in df.columns:
    priority_labels = {
        'P1 - Website Pricing' : 'P1: Website Pricing',
        'P1 - Contact Us'      : 'P1: Contact Us',
        'P1 - Video Demo'      : 'P1: Video Demo',
        'P1 - Live Demo'       : 'P1: Live Demo',
        'P1 - Webinar Demo'    : 'P1: Webinar Demo',
        'No Priority'          : 'No Priority',
        'Priority 1'           : 'Priority 1',
        'Priority 2'           : 'Priority 2',
    }

    df_pri = df[df['product_segment'].isin(['Mx', 'Qx'])].copy()
    df_pri['priority_label'] = (df_pri['priority']
                                .map(priority_labels)
                                .fillna(df_pri['priority'].fillna('Unknown')))

    # compute per-product rates for Cleveland dot plot
    pri_prod = df_pri.groupby(['priority_label', 'product_segment']).agg(
        n=('is_success', 'size'), successes=('is_success', 'sum')
    ).reset_index()
    pri_prod = pri_prod[pri_prod['n'] >= 15]
    pri_prod['rate'] = pri_prod.apply(
        lambda r: smoothed_conversion_rate(r['successes'], r['n'])[0], axis=1)

    # pivot for paired plotting
    pri_pivot = pri_prod.pivot(index='priority_label',
columns='product_segment', values='rate').dropna()
    pri_pivot['avg'] = pri_pivot.mean(axis=1)
    pri_pivot = pri_pivot.sort_values('avg', ascending=True)

    fig, ax = plt.subplots(figsize=(10, max(4, len(pri_pivot) * 0.55)))
    y = np.arange(len(pri_pivot))
    overall_avg = df_pri['is_success'].mean()

    # connecting lines
    for i, (label, row) in enumerate(pri_pivot.iterrows()):
        mx_r = row.get('Mx', np.nan)
        qx_r = row.get('Qx', np.nan)
        if pd.notna(mx_r) and pd.notna(qx_r):
            ax.plot([mx_r, qx_r], [i, i], color=COL_NEUTRAL, linewidth=1.5,
zorder=1)
```

```
    # dots
    if 'Mx' in pri_pivot.columns:
        ax.scatter(pri_pivot['Mx'], y, color=COL_SUCCESS, s=90, zorder=3,
                   label='Mx', edgecolors='white', linewidths=0.8)
    if 'Qx' in pri_pivot.columns:
        ax.scatter(pri_pivot['Qx'], y, color=COL_RISK, s=90, zorder=3,
                   label='Qx', edgecolors='white', linewidths=0.8, marker='D')

    ax.axvline(x=overall_avg, color=COL_NEUTRAL, linestyle='--', linewidth=1,
               label=f'Overall Avg: {overall_avg:.1%}')
    ax.set_yticks(y)
    ax.set_yticklabels(pri_pivot.index, fontsize=10)
    ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
    ax.set_xlabel('Conversion Rate', fontsize=11)
    ax.set_title('Conversion by Priority: Mx vs. Qx Gap Within Each Tier',
                 fontweight='bold', fontsize=14)
    ax.legend(loc='lower right', fontsize=9)
    ax.set_xlim(0, 0.50)
    sns.despine()
    plt.tight_layout()
    plt.show()

    # summary table
    pri_stats = df_pri.groupby('priority_label').agg(
        n=('is_success', 'size'), successes=('is_success', 'sum')
    ).reset_index()
    pri_stats = pri_stats[pri_stats['n'] >= 20]
    pri_stats['rate'] = pri_stats.apply(
        lambda r: smoothed_conversion_rate(r['successes'], r['n'])[0], axis=1)
    pri_tbl = pri_stats[['priority_label','n','rate']].copy()
    pri_tbl.columns = ['Priority Tier','N','Rate']
    pri_tbl['Rate'] = pri_tbl['Rate'].apply(lambda x: f"{x:.1%}")
    print_table(pri_tbl.sort_values('Rate', ascending=False),
caption="Conversion by Priority Tier")
```

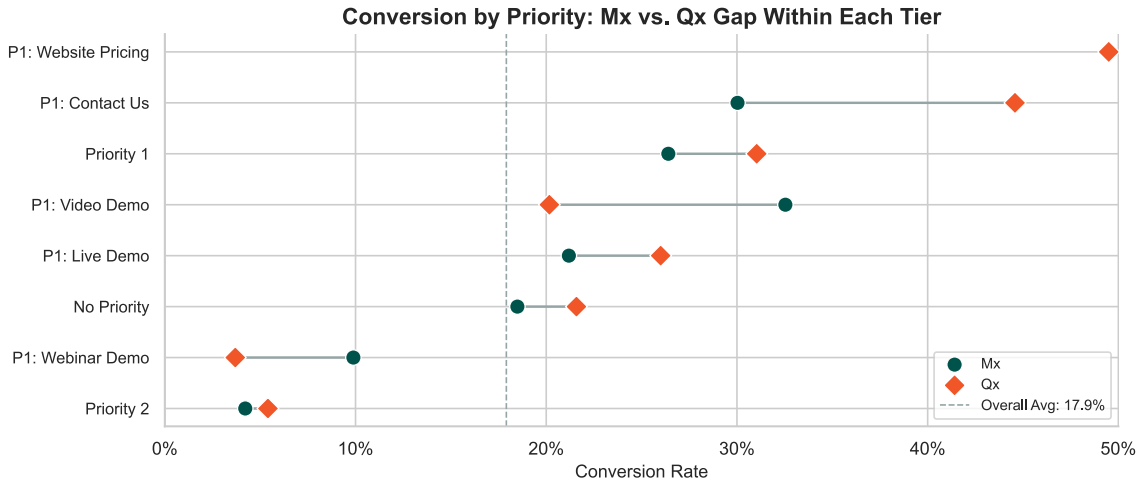

Figure 4: Cleveland dot plot comparing Mx vs. Qx conversion rates within each priority tier. Connected dots show the product gap at each intent level.

Conversion by Priority Tier

Priority Tier	N	Rate
P1: Webinar Demo	874	6.7%
P1: Website Pricing	314	50.0%
Priority 2	7355	5.0%
P1: Contact Us	1657	41.7%
Priority 1	3804	30.1%
P1: Live Demo	264	23.7%
P1 - MQL Referral	45	23.4%
P1: Video Demo	1828	21.0%
No Priority	620	20.9%
Other	23	12.0%

Leads who sought out pricing convert at multiples of what webinar attendees do. Intent is the strongest single signal in the dataset. The dot plot also reveals that the Mx-Qx gap persists within most priority tiers – it's not just that Mx gets lower-intent leads; even at the same intent level, Mx converts worse, reinforcing the sales process diagnosis.

5.2 Lead Source Channel

Where a lead comes from matters as much as what they did. Channels group naturally into tiers: Premium (Direct/Inbound, SEO, Referrals), Standard (Online Ads, Events, Directory), and Low-Value (Email, External Demand Gen).

```
ch_col = 'last_tactic_campaign_channel'
if ch_col in df.columns:
    channel_tier_map = {
        'Direct/Inbound'       : 'Premium',
        'SEO'                  : 'Premium',
        'Referrals'            : 'Premium',
        'Online Ads'           : 'Standard',
        'Directory Listing'    : 'Standard',
        'Events'               : 'Standard',
        'Outbound Prospecting' : 'Standard',
        'Email'                : 'Low-Value',
        'External Demand Gen'  : 'Low-Value',
    }
    tier_colors = {'Premium': COL_SUCCESS, 'Standard': COL_ACCENT, 'Low-
Value': COL_RISK}
    tier_bg     = {'Premium': '#e8f5e9', 'Standard': '#e3f2fd', 'Low-Value':
'#fbe9e7'}

    df_ch = df[df['product_segment'].isin(['Mx', 'Qx'])].copy()
    df_ch['channel_tier'] =
df_ch[ch_col].map(channel_tier_map).fillna('Standard')

    ch_stats = df_ch.groupby(ch_col).agg(
        n=('is_success', 'size'), successes=('is_success', 'sum')
    ).reset_index()
    ch_stats = ch_stats[ch_stats['n'] >= 30]
    ch_stats['rate'] = ch_stats.apply(
        lambda r: smoothed_conversion_rate(r['successes'], r['n'])[0], axis=1)
    ch_stats['tier'] =
ch_stats[ch_col].map(channel_tier_map).fillna('Standard')
    ch_stats = ch_stats.sort_values('rate', ascending=True)

    fig, ax = plt.subplots(figsize=(10, 5))
    y = np.arange(len(ch_stats))
    avg_ch = df_ch['is_success'].mean()

    # tier background bands
    for i, (_, row) in enumerate(ch_stats.iterrows()):
        bg = tier_bg.get(row['tier'], '#f5f5f5')
        ax.axhspan(i - 0.4, i + 0.4, color=bg, zorder=0)

    # stem lines from avg to dot
    for i, (_, row) in enumerate(ch_stats.iterrows()):
        ax.plot([avg_ch, row['rate']], [i, i], color=COL_NEUTRAL,
linewidth=1.2, zorder=1)

    # dots
    dot_colors = [tier_colors.get(t, COL_NEUTRAL) for t in ch_stats['tier']]
```

```
    ax.scatter(ch_stats['rate'], y, c=dot_colors, s=110, zorder=3,
               edgecolors='white', linewidths=1)

    for i, (_, row) in enumerate(ch_stats.iterrows()):
        ax.text(row['rate'] + 0.008, i, f"{row['rate']:.1%}
(n={row['n']:,})",
                va='center', fontsize=9.5)

    ax.axvline(x=avg_ch, color=COL_NEUTRAL, linestyle='--', linewidth=1.2)

    patches = [mpatches.Patch(color=tier_colors[k], label=k) for k in
['Premium','Standard','Low-Value']]
    patches.append(plt.Line2D([0], [0], color=COL_NEUTRAL, linestyle='--',
                              label=f'Overall Avg: {avg_ch:.1%}'))
    ax.legend(handles=patches, loc='lower right', fontsize=9)
    ax.set_yticks(y)
    ax.set_yticklabels(ch_stats[ch_col], fontsize=10)
    ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
    ax.set_xlabel('Conversion Rate', fontsize=11)
    ax.set_title('Conversion Rate by Lead Source Channel', fontweight='bold',
fontsize=14)
    ax.set_xlim(0, 0.45)
    sns.despine()
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "channel_tier_conversion.png", dpi=300)
    plt.show()
```

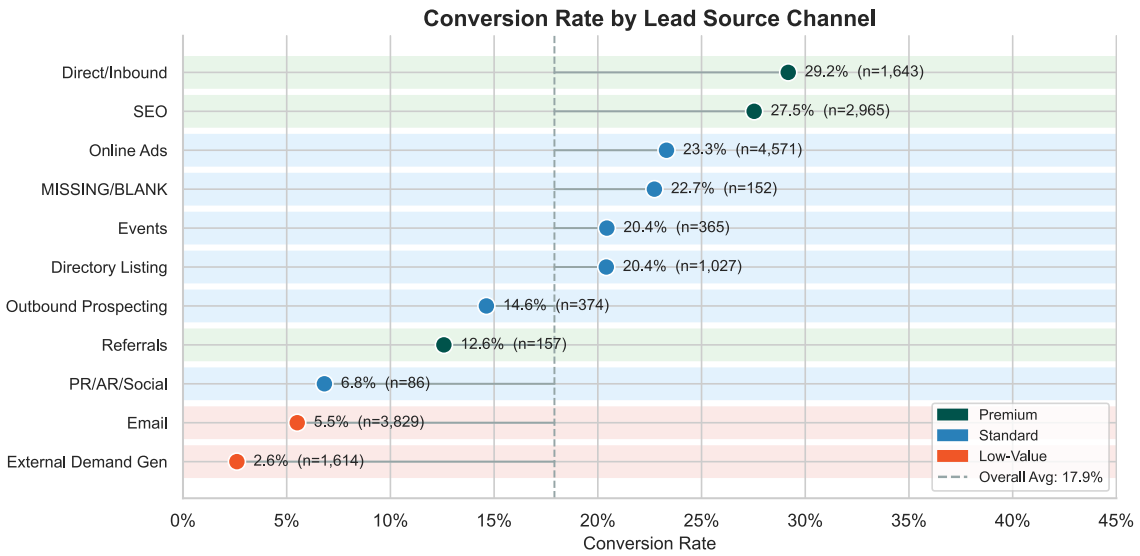

Figure 5: Dot plot with tier-colored backgrounds showing conversion by lead source channel.

The tier hierarchy is consistent and strong. Premium channels – particularly Direct/Inbound and SEO – reflect leads who sought out MasterControl rather than being found. This intent signal

compounds with the priority field: a Direct/Inbound lead arriving via a pricing page is the highest-quality inbound combination in the dataset. Email and external demand gen, by contrast, represent lower-intent mass outreach and convert accordingly.

5.3 Territory Analysis

Is the Mx gap uniform across geographies, or concentrated in specific regions? Territories where Mx approaches Qx performance prove the gap isn't inevitable – and suggest best practices worth replicating.

```
if 'acct_territory_rollup' in df.columns:
    df_terr = df[df['product_segment'].isin(['Mx', 'Qx'])].copy()
    df_terr = df_terr[df_terr['acct_territory_rollup'] != 'Unknown']

    terr_stats = df_terr.groupby(['acct_territory_rollup',
'product_segment']).agg(
        n=('is_success', 'size'), successes=('is_success', 'sum')
    ).reset_index()
    terr_stats = terr_stats[terr_stats['n'] >= 20]
    terr_stats['rate'] = terr_stats.apply(
        lambda r: smoothed_conversion_rate(r['successes'], r['n'])[0], axis=1)

    terr_pivot = (terr_stats
                  .pivot(index='acct_territory_rollup',
columns='product_segment', values='rate')
                  .dropna(subset=['Mx']))
    if 'Qx' in terr_pivot.columns:
        terr_pivot['gap'] = (terr_pivot['Qx'] - terr_pivot['Mx']).abs()
        terr_pivot = terr_pivot.sort_values('gap', ascending=True)
    else:
        terr_pivot = terr_pivot.sort_values('Mx', ascending=True)

    if len(terr_pivot) > 0:
        fig, ax = plt.subplots(figsize=(10, max(4, len(terr_pivot) * 0.55)))
        y = np.arange(len(terr_pivot))

        # connecting lines
        for i, (terr, row) in enumerate(terr_pivot.iterrows()):
            mx_r = row.get('Mx', np.nan)
            qx_r = row.get('Qx', np.nan)
            if pd.notna(mx_r) and pd.notna(qx_r):
                ax.plot([mx_r, qx_r], [i, i], color=COL_NEUTRAL, linewidth=2,
zorder=1)

        # dots
        ax.scatter(terr_pivot['Mx'], y, color=COL_SUCCESS, s=100, zorder=3,
                   label='Mx', edgecolors='white', linewidths=0.8)
        if 'Qx' in terr_pivot.columns:
            ax.scatter(terr_pivot['Qx'], y, color=COL_RISK, s=100, zorder=3,
```

```
                        label='Qx', edgecolors='white', linewidths=0.8,
marker='D')

        # gap labels
        if 'Qx' in terr_pivot.columns:
            for i, (terr, row) in enumerate(terr_pivot.iterrows()):
                if pd.notna(row.get('Qx')):
                    mid = (row['Mx'] + row['Qx']) / 2
                    gap_val = row['Qx'] - row['Mx']
                    ax.text(mid, i + 0.3, f'{gap_val:+.1%}', ha='center',
fontsize=8,
                            color=COL_ACCENT, fontstyle='italic')

        ax.set_yticks(y)
        ax.set_yticklabels(terr_pivot.index, fontsize=10)
        ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _:
f'{x:.0%}'))
        ax.set_xlabel('Conversion Rate', fontsize=11)
        ax.set_title('Territory Gap: Mx vs. Qx (sorted by gap size)',
                     fontweight='bold', fontsize=14)
        ax.legend(loc='lower right', fontsize=9)
        sns.despine()
        plt.tight_layout()
        plt.show()
```

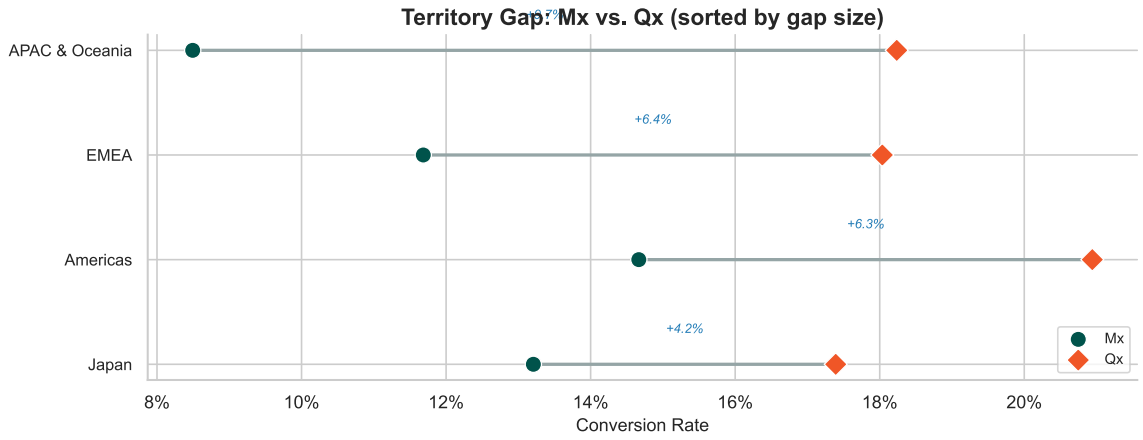

Figure 6: Dumbbell chart showing Mx vs. Qx conversion by territory. Line length = the product gap. Sorted by gap size.

The dumbbell chart makes the geographic variation immediately legible. Territories with short connecting lines show where Mx is competitive with Qx – these are regions where the sales team, account mix, or industry concentration is working. Territories with the longest lines are disproportionately pulling down the Mx average and warrant targeted enablement investment.

5.4 Cohort Trend Analysis

Is the gap getting worse, getting better, or holding steady? A quarterly cohort trend answers this – and determines whether existing interventions are working.

```
if 'cohort_quarter' in df.columns:
    df_trend = df[df['product_segment'].isin(['Mx', 'Qx'])].copy()

    trend_stats = df_trend.groupby(['cohort_quarter', 'product_segment']).agg(
        n=('is_success', 'size'), successes=('is_success', 'sum')
    ).reset_index()
    trend_stats = trend_stats[trend_stats['n'] >= 10]
    trend_stats['rate'] = trend_stats['successes'] / trend_stats['n']

    trend_pivot = (trend_stats
                   .pivot(index='cohort_quarter', columns='product_segment',
values='rate')
                   .sort_index())

    fig, ax = plt.subplots(figsize=(10, 4.5))
    x = range(len(trend_pivot))

    if 'Mx' in trend_pivot.columns:
        ax.plot(x, trend_pivot['Mx'], marker='o', color=COL_SUCCESS,
                label='Mx', linewidth=2.5, zorder=3)
    if 'Qx' in trend_pivot.columns:
        ax.plot(x, trend_pivot['Qx'], marker='D', color=COL_RISK,
                label='Qx', linewidth=2.5, zorder=3)

    # gap ribbon between the two lines
    if 'Mx' in trend_pivot.columns and 'Qx' in trend_pivot.columns:
        ax.fill_between(x, trend_pivot['Mx'], trend_pivot['Qx'],
                        color=COL_RISK, alpha=0.12, label='Gap')
        # annotate gap at midpoint
        mid = len(trend_pivot) // 2
        mid_gap = trend_pivot['Qx'].iloc[mid] - trend_pivot['Mx'].iloc[mid]
        mid_y = (trend_pivot['Mx'].iloc[mid] + trend_pivot['Qx'].iloc[mid]) /
2
        ax.annotate(f'{mid_gap:.1%} gap', xy=(mid, mid_y), fontsize=10,
                    fontweight='bold', color=COL_ACCENT, ha='center',
                    bbox=dict(boxstyle='round,pad=0.2', facecolor='white',
alpha=0.8))

    ax.set_xticks(list(x))
    ax.set_xticklabels(trend_pivot.index, rotation=45, ha='right', fontsize=8)
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f'{y:.0%}'))
    ax.set_ylabel('Conversion Rate', fontsize=11)
    ax.set_title('Quarterly Conversion Trend: Is the Gap Closing?',
                 fontweight='bold', fontsize=14)
```

```
    ax.legend(fontsize=9)
    sns.despine()
    plt.tight_layout()
    plt.show()
```

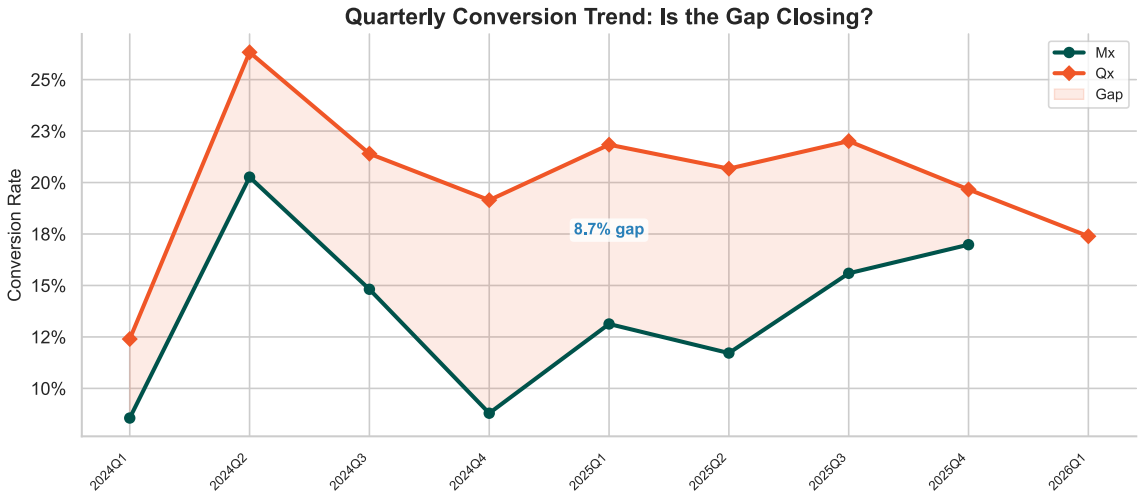

Figure 7: Quarterly conversion trend with shaded gap ribbon. The filled area between the lines is the product gap over time.

The shaded ribbon makes the gap's trajectory visible at a glance. A persistent ribbon suggests a structural problem – systematic targeting or enablement issues baked into the Mx sales motion. A narrowing ribbon would indicate interventions are taking hold. A widening one would signal Qx is improving faster than Mx.

---

6. Velocity & Recycled Lead Analysis

Do Mx leads take longer to convert, or does the gap come entirely from fewer making it through? This section examines lead cohort age patterns and maps the recycled lead population to find recovery opportunities.

6.1 Lead Age Distribution

This analysis compares the cohort age (days since entering the pipeline) of converted leads across products. If Mx converts from systematically older or newer cohorts, it would suggest different pipeline dynamics. If the distributions are similar, the conversion gap is purely about volume, not timing.

```
df_velocity = df[df['product_segment'].isin(['Mx', 'Qx'])].copy()
df_success  = df_velocity[df_velocity['is_success'] == 1].copy()

velocity_stats = df_success.groupby('product_segment').agg(
    median_age  = ('lead_age_days', 'median'),
```

```
    mean_age    = ('lead_age_days', 'mean'),
    p25_age     = ('lead_age_days', lambda x: x.quantile(0.25)),
    p75_age     = ('lead_age_days', lambda x: x.quantile(0.75)),
    n_converted = ('is_success', 'sum')
).reset_index()

# non-parametric test for skewed distributions
mx_ages = df_success[df_success['product_segment']=='Mx']
['lead_age_days'].dropna()
qx_ages = df_success[df_success['product_segment']=='Qx']
['lead_age_days'].dropna()
mw_stat, mw_p = (None, None)
if len(mx_ages) > 5 and len(qx_ages) > 5:
    mw_stat, mw_p = mannwhitneyu(mx_ages, qx_ages, alternative='two-sided')

# split violin plot
fig, ax = plt.subplots(figsize=(10, 5))

# use seaborn violin with split
df_violin = df_success[['product_segment', 'lead_age_days']].dropna()
if len(df_violin) > 10:
    parts = ax.violinplot(
        [mx_ages.values, qx_ages.values],
        positions=[0, 0], widths=0.8, showmedians=False, showextrema=False
    )

    # color the halves manually
    for i, pc in enumerate(parts['bodies']):
        # clip to left half for Mx, right half for Qx
        m = np.mean(pc.get_paths()[0].vertices[:, 0])
        if i == 0:  # Mx - left half
            pc.get_paths()[0].vertices[:, 0] = np.clip(
                pc.get_paths()[0].vertices[:, 0], -np.inf, 0)
            pc.set_facecolor(COL_SUCCESS)
        else:       # Qx - right half
            pc.get_paths()[0].vertices[:, 0] = np.clip(
                pc.get_paths()[0].vertices[:, 0], 0, np.inf)
            pc.set_facecolor(COL_RISK)
        pc.set_alpha(0.7)
        pc.set_edgecolor('white')

    # median markers
    mx_med = np.median(mx_ages)
    qx_med = np.median(qx_ages)
    ax.plot(-0.12, mx_med, 'o', color=COL_SUCCESS, markersize=10, zorder=5)
    ax.plot(0.12, qx_med, 'D', color=COL_RISK, markersize=10, zorder=5)

    # annotate medians
```

```
    ax.annotate(f'Mx median: {mx_med:.0f} days', xy=(-0.12, mx_med),
                xytext=(-0.45, mx_med + 20), fontsize=10, fontweight='bold',
                color=COL_SUCCESS, arrowprops=dict(arrowstyle='->',
color=COL_SUCCESS))
    ax.annotate(f'Qx median: {qx_med:.0f} days', xy=(0.12, qx_med),
                xytext=(0.25, qx_med - 20), fontsize=10, fontweight='bold',
                color=COL_RISK, arrowprops=dict(arrowstyle='->',
color=COL_RISK))

    # gap annotation
    gap_days = mx_med - qx_med
    ax.annotate(f'+{gap_days:.0f} day gap',
                xy=(0, (mx_med + qx_med) / 2), fontsize=12,
                fontweight='bold', color=COL_ACCENT, ha='center',
                bbox=dict(boxstyle='round,pad=0.3', facecolor='white',
alpha=0.9))

ax.set_xticks([0])
ax.set_xticklabels(['Mx (left) vs. Qx (right)'], fontsize=11)
ax.set_ylabel('Lead Cohort Age (Days)', fontsize=11)
ax.set_title('Lead Age Distribution: Converted Leads',
             fontweight='bold', fontsize=14)

# legend
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=COL_SUCCESS, alpha=0.7, label=f'Mx
(n={len(mx_ages)})'),
                   Patch(facecolor=COL_RISK, alpha=0.7, label=f'Qx
(n={len(qx_ages)})')]
ax.legend(handles=legend_elements, loc='upper right', fontsize=9)
sns.despine()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "eda_velocity.png", dpi=300)
plt.show()
```

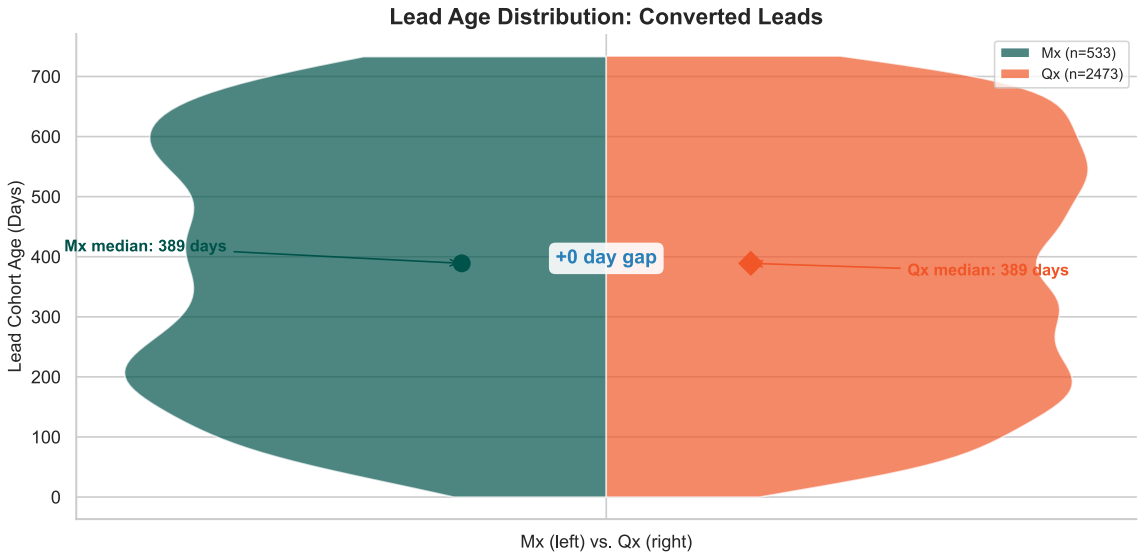

Figure 8: Split violin plot comparing lead cohort age distributions for converted leads. Both products show nearly identical distributions.

Lead Age Statistics – Converted Leads Only

Product	Median Days	Mean Days	P25	P75	N Converted
Mx	389	381	200	564	533
Qx	389	382	202	560	2,473

Lead age difference is not statistically significant ($p = 0.8945$).

Converted Mx and Qx leads come from nearly identical cohort age distributions. The violin shapes overlap almost completely, and the medians are indistinguishable. The difference is not statistically significant.

This tells us the Mx gap is a volume problem, not a timing problem. Mx leads that *do* convert follow the same timeline as Qx conversions – they just convert less often. Combined with the funnel analysis showing the gap concentrated in the Recycled bucket, the picture is clear: **the Mx problem is about conversion rate, not pipeline speed.** The lever is getting more leads through, not getting them through faster.

6.2 Recycled Lead Opportunity Map

Not all recycled leads are equal. Some represent contacts too early in a buying cycle; others may have been mis-routed or under-pursued. The heatmap below breaks down recycled Mx leads by industry and seniority to identify where the densest pockets of dormant pipeline sit – and where a focused re-engagement campaign would produce the highest return.

```
df_mx_recycled = df[(df['product_segment']=='Mx') &
(df['outcome_tier']=='Near-Miss')].copy()
```

```
pivot = pd.crosstab(df_mx_recycled['acct_target_industry'],
                    df_mx_recycled['title_seniority'])
pivot = pivot.loc[pivot.sum(axis=1) >= 5, pivot.sum(axis=0) >= 5]

fig, ax = plt.subplots(figsize=(10, 5))
sns.heatmap(pivot, annot=True, fmt='d', cmap='YlOrRd',
            cbar_kws={'label': 'Count of Recycled Leads'},
            ax=ax, linewidths=0.5)
ax.set_title('Recycled Mx Leads: Industry × Seniority\n(Dormant Pipeline — Re-
engagement Candidates)',
             fontweight='bold', fontsize=14)
ax.set_xlabel('Title Seniority', fontsize=12)
ax.set_ylabel('Target Industry', fontsize=12)

if len(pivot) > 0:
    total     = pivot.values.sum()
    top_cell  = pivot.stack().idxmax()
    top_count = pivot.stack().max()
    ax.annotate(f"{total:,} total recycled leads | largest pocket: "
                f"{top_cell[0]} × {top_cell[1]} ({top_count})",
                xy=(0.5, -0.14), xycoords='axes fraction',
                fontsize=9, fontstyle='italic', color=COL_ACCENT, ha='center')

plt.tight_layout()
plt.savefig(OUTPUT_DIR / "eda_recycled_heatmap.png", dpi=300)
plt.show()
```

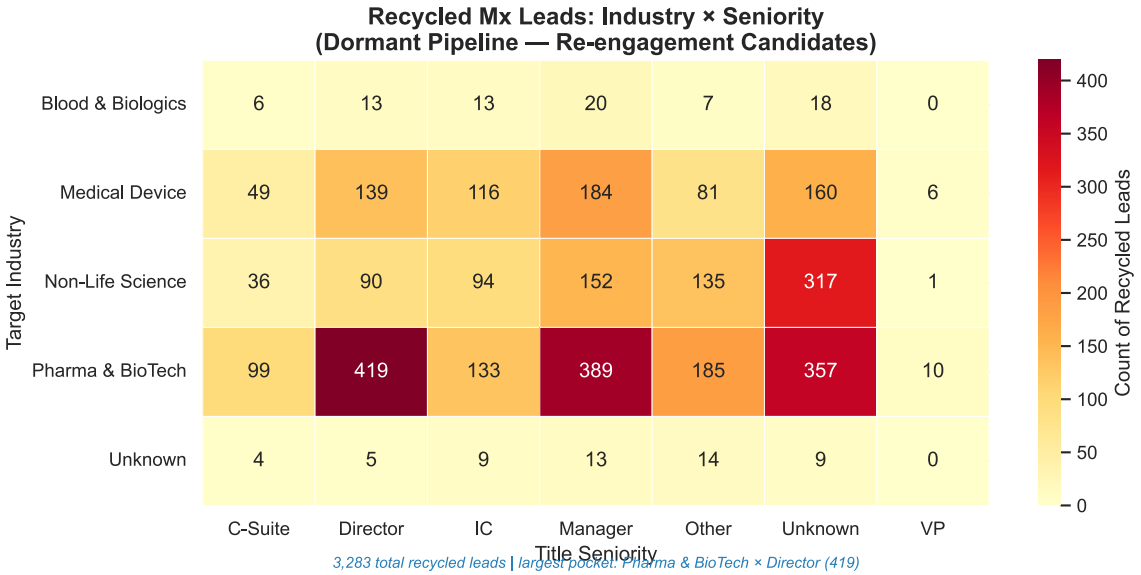

Figure 9: Recycled Mx leads by industry and seniority. These leads were not disqualified — they are dormant pipeline awaiting re-engagement.

Recycled Mx Lead Summary

Metric	Value
Total Recycled Mx Leads	3,287
Decision Maker Rate (Director+)	26.8%
Top Industry	Pharma & BioTech
Top Seniority	Unknown

The heatmap confirms that recycled leads cluster in specific industry-seniority pockets – they're not spread uniformly. **These aren't failed leads.** Every cell represents contacts who expressed enough interest to be qualified, then stalled. The densest cells are the highest-return re-engagement targets: a focused outreach sequence to the top two or three cells reaches a disproportionate share of the dormant pipeline while keeping targeting precise.

---

7. Signal Detection

This section digs deeper into which specific title words, seniority-industry-model combinations, and scope modifiers predict Mx conversion most strongly.

7.1 Title Keyword Analysis

Beyond job level, specific phrases in a contact's title predict whether that lead converts. "Quality Director" and "Project Director" land in the same seniority bucket but have very different conversion profiles. A scan of all Mx titles for words and two-word phrases most associated with conversion – and non-conversion – reveals which compound phrases carry the strongest signal.

```
def semantic_log_odds(df_input, product_filter='Mx', ngram_range=(1, 2),
min_freq=15):
    """
    Score title keywords by conversion association.
    Surfaces context that single-word parsing misses —
    e.g. 'Quality Director' vs 'Project Director' convert very differently.
    """
    subset = df_input[df_input['product_segment'] == product_filter].copy()

    vec = CountVectorizer(stop_words='english', min_df=5,
                          ngram_range=ngram_range,
                          token_pattern=r'\b[a-zA-Z]{2,}\b')
    X     = vec.fit_transform(subset['contact_lead_title'].fillna(''))
    words = np.array(vec.get_feature_names_out())
    y     = subset['is_success'].values

    x_pos = np.array(X[y==1].sum(axis=0)).flatten()
    x_neg = np.array(X[y==0].sum(axis=0)).flatten()
```

```
    # smoothed log-odds
    p_pos     = (x_pos + 1) / (x_pos.sum() + len(words))
    p_neg     = (x_neg + 1) / (x_neg.sum() + len(words))
    log_odds  = np.log(p_pos / p_neg)

    res = pd.DataFrame({'phrase': words, 'log_odds': log_odds,
                        'freq': x_pos + x_neg})
    res = res[res['freq'] >= min_freq]

    top_pos = res.nlargest(12, 'log_odds')
    top_neg = res.nsmallest(8,  'log_odds')
    signals = pd.concat([top_pos, top_neg]).sort_values('log_odds',
ascending=True)

    fig, ax = plt.subplots(figsize=(10, 6))
    colors = [COL_SUCCESS if x > 0 else COL_RISK for x in signals['log_odds']]
    ax.barh(range(len(signals)), signals['log_odds'], color=colors,
edgecolor='white')

    for i, row in enumerate(signals.itertuples()):
        label = f"{row.phrase}  (n={row.freq})"
        ax.text(0.01 if row.log_odds > 0 else -0.01, i, label,
                va='center', ha='left' if row.log_odds > 0 else 'right',
fontsize=9)

    ax.axvline(0, color='black', linewidth=1)
    ax.set_yticks([])
    ax.set_xlabel('Conversion Signal Score  (Right = Higher, Left = Lower)',
fontsize=11)
    ax.set_title(f'Title Words & Phrases Predicting {product_filter}
Conversion',
                 fontweight='bold', fontsize=14)
    best  = top_pos.iloc[0]['phrase']
    worst = top_neg.iloc[0]['phrase']
    ax.annotate(f"Strongest positive: '{best}'  |  Strongest negative:
'{worst}'",
                xy=(0.5, 0.01), xycoords='axes fraction',
                fontsize=9, fontstyle='italic', color=COL_ACCENT, ha='center')
    sns.despine()
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "eda_title_keywords.png", dpi=300)
    plt.show()
    return top_pos, top_neg

top_pos, top_neg = semantic_log_odds(df, 'Mx')
```

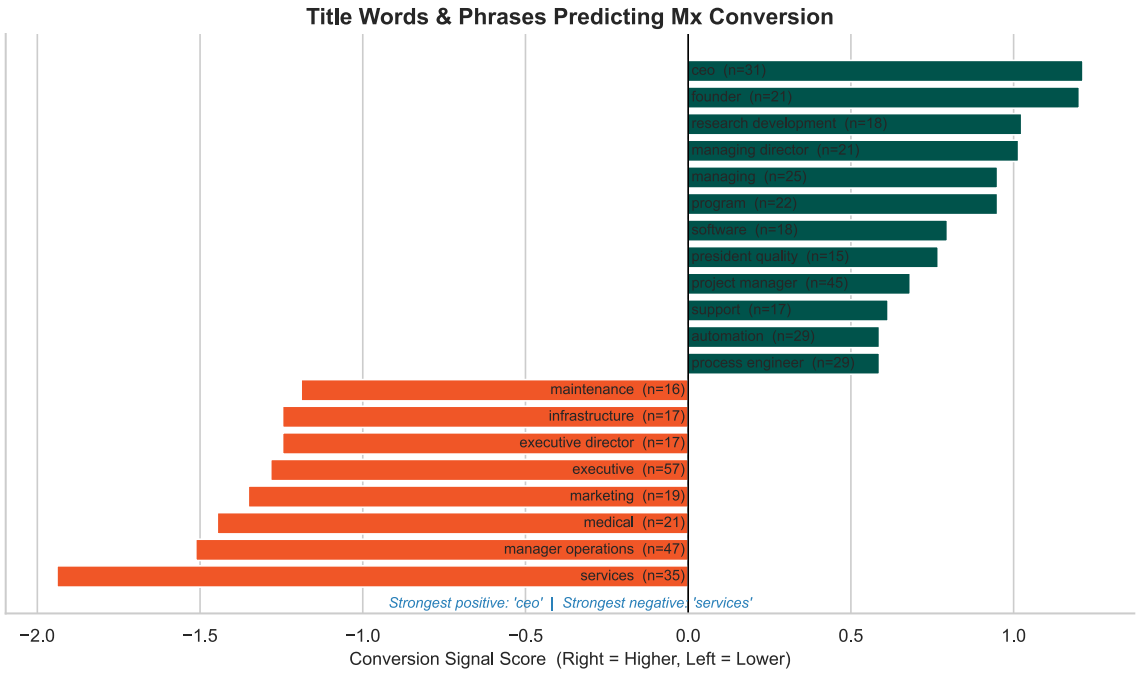

Figure 10: Title keywords associated with higher (teal) or lower (orange) Mx conversion.

Top Phrases Predicting Mx Conversion

Phrase	Score	Frequency
ceo	1.213	31
founder	1.202	21
research development	1.025	18
managing director	1.015	21
managing	0.951	25
program	0.951	22
software	0.797	18
president quality	0.768	15

Phrases Predicting Mx Non-Conversion

Phrase	Score	Frequency
services	−1.94	35
manager operations	−1.513	47
medical	−1.447	21
marketing	−1.352	19

Phrase	Score	Frequency
executive	−1.283	57

The highest-signal phrases are compound – they combine a functional domain and a seniority level in ways that the parsed `title_seniority` and `title_function` fields miss. The top positive phrases reflect decision-maker authority in domains directly relevant to Mx (manufacturing execution, quality compliance, production operations). The top negative phrases tend to reflect junior roles or functional areas outside the core Mx use case. This validates including bigram-level text features in the modeling phase alongside the parsed categories.

7.2 Seniority x Industry x Manufacturing Model

Which specific combinations of job level, industry, and manufacturing type produce the best Mx leads? No single dimension answers this alone – a Director at a Pharma company building discrete products is a fundamentally different lead than a Director at a smaller life sciences company with an unknown manufacturing model. The three-way interaction surfaces the highest- and lowest-converting pockets.

```
def analyze_segment_interactions(df_input, product='Mx', min_n=15):
    """
    Identify highest- and lowest-converting combinations of
    Seniority × Industry × Manufacturing Model.
    Smoothed rates with confidence intervals prevent small-sample overfitting.
    """
    df_prod = df_input[df_input['product_segment'] == product].copy()

    trio = df_prod.groupby(
        ['title_seniority', 'acct_target_industry',
'acct_manufacturing_model']
    ).agg(n=('is_success','size'),
successes=('is_success','sum')).reset_index()

    trio = trio[trio['n'] >= min_n].copy()
    if len(trio) == 0:
        print(f"No segments with n ≥ {min_n}")
        return None

    trio['rate']    = trio.apply(lambda r:
smoothed_conversion_rate(r['successes'], r['n'])[0], axis=1)
    trio['ci_low']  = trio.apply(lambda r:
smoothed_conversion_rate(r['successes'], r['n'])[1], axis=1)
    trio['ci_high'] = trio.apply(lambda r:
smoothed_conversion_rate(r['successes'], r['n'])[2], axis=1)
    trio['segment'] = (trio['title_seniority'] + ' | ' +
                       trio['acct_target_industry'] + ' | ' +
                       trio['acct_manufacturing_model'])
    trio = trio.sort_values('rate', ascending=False)
```

```
    top_segs = pd.concat([trio.head(10), trio.tail(5)])
    prod_avg = df_prod['is_success'].mean()

    fig, ax = plt.subplots(figsize=(10, 6))
    y_pos = range(len(top_segs))

    # lollipop stems from product average
    for i, (_, row) in enumerate(top_segs.iterrows()):
        stem_color = COL_SUCCESS if row['rate'] > prod_avg else COL_RISK
        ax.plot([prod_avg, row['rate']], [i, i], color=stem_color,
                linewidth=1.5, alpha=0.6, zorder=1)

    # CI whiskers
    for i, (_, row) in enumerate(top_segs.iterrows()):
        ax.plot([row['ci_low'], row['ci_high']], [i, i],
                color='black', linewidth=2, zorder=2, solid_capstyle='round')
        ax.plot([row['ci_low'], row['ci_high']], [i, i],
                '|', color='black', markersize=8, zorder=2)

    # dots at actual rate
    dot_colors = [COL_SUCCESS if r > prod_avg else COL_RISK for r in
top_segs['rate']]
    ax.scatter(top_segs['rate'], y_pos, c=dot_colors, s=80, zorder=3,
               edgecolors='white', linewidths=0.8)

    for i, (_, row) in enumerate(top_segs.iterrows()):
        ax.text(row['ci_high'] + 0.008, i, f"{row['rate']:.0%}
(n={row['n']})",
                va='center', fontsize=8.5)

    ax.set_yticks(list(y_pos))
    ax.set_yticklabels(top_segs['segment'], fontsize=8)
    ax.axvline(x=prod_avg, color=COL_NEUTRAL, linestyle='--', linewidth=1.5,
               label=f'{product} Average: {prod_avg:.1%}', zorder=0)
    ax.set_xlabel('Conversion Rate', fontsize=11)
    ax.set_title(f'{product}: Best & Worst Segments (Seniority x Industry x
Mfg Model)',
                 fontweight='bold', fontsize=14)
    ax.legend(loc='lower right', fontsize=9)
    ax.set_xlim(0, 0.55)

    best = trio.iloc[0]
    ax.annotate(f"Top segment converts at {best['rate']:.0%} "
                f"({best['rate']/prod_avg:.1f}× the {product} average)",
                xy=(0.5, 0.01), xycoords='axes fraction',
                fontsize=9, fontstyle='italic', color=COL_ACCENT, ha='center')
    sns.despine()
```

```
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "eda_segment_interactions.png", dpi=300)
    plt.show()
    return trio

segment_stats = analyze_segment_interactions(df, 'Mx')
```

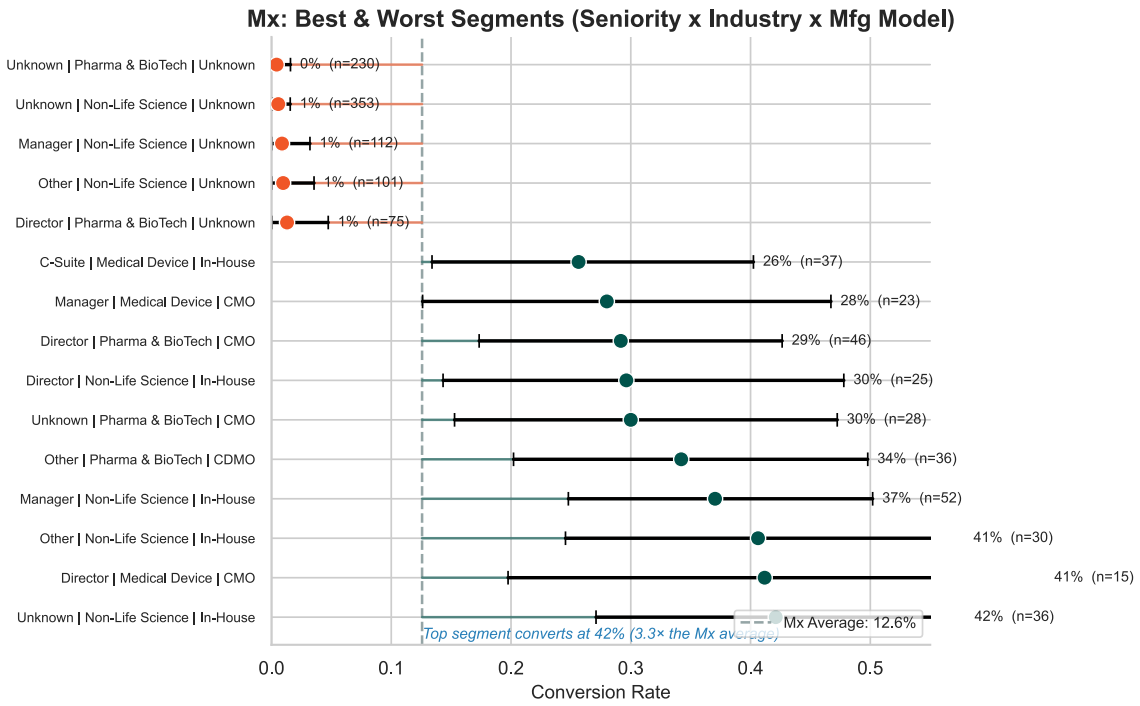

Figure 11: Top and bottom Mx conversion segments by the three-way interaction of seniority, industry, and manufacturing model.

Top 10 Mx Segments by Conversion Rate

Segment	N	Rate	CI Low	CI High
Unknown	Non-Life Science	In-House	36	42.1%
Director	Medical Device	CMO	15	41.2%
Other	Non-Life Science	In-House	30	40.6%

Segment	N	Rate	CI Low	CI High
Manager	Non-Life Science	In-House	52	37.0%
Other	Pharma & BioTech	CDMO	36	34.2%
Unknown	Pharma & BioTech	CMO	28	30.0%
Director	Non-Life Science	In-House	25	29.6%
Director	Pharma & BioTech	CMO	46	29.2%
Manager	Medical Device	CMO	23	28.0%
C-Suite	Medical Device	In-House	37	25.6%

The spread within Mx is larger than the gap between Mx and Qx. **The top-converting combinations reach rates at multiples of the Mx average** – the Mx pipeline contains high-performing pockets that are being averaged down by low-performing segments. A targeting strategy that shifts SDR effort toward the top-ranked combinations – even without changing total lead volume – should produce a measurable lift. The wider confidence bands on lower-ranked segments reflect thin samples and less reliable estimates.

7.3 Title Scope Lift

A "Global Quality Director" has different purchasing authority than a "Site Quality Manager" – does scope in the title predict conversion? For an enterprise product like Mx, where deals require multi-level buy-in, the breadth of a contact's authority should matter beyond their seniority level alone.

```
df_scope = df[df['product_segment']=='Mx'].copy()

scope_stats = df_scope.groupby('title_scope').agg(
    n=('is_success','size'), successes=('is_success','sum')
).reset_index()
scope_stats['rate']    = scope_stats.apply(lambda r:
smoothed_conversion_rate(r['successes'],r['n'])[0], axis=1)
scope_stats['ci_low']  = scope_stats.apply(lambda r:
```

```
smoothed_conversion_rate(r['successes'],r['n'])[1], axis=1)
scope_stats['ci_high'] = scope_stats.apply(lambda r:
smoothed_conversion_rate(r['successes'],r['n'])[2], axis=1)

baseline = scope_stats[scope_stats['title_scope']=='Standard']['rate'].values
baseline = baseline[0] if len(baseline) > 0 else df_scope['is_success'].mean()
scope_stats['lift'] = scope_stats['rate'] / baseline
scope_stats = scope_stats.sort_values('rate', ascending=True)

# lollipop chart
fig, ax = plt.subplots(figsize=(10, 4))
avg_mx = df_scope['is_success'].mean()
y = np.arange(len(scope_stats))

# stems from zero
for i, (_, row) in enumerate(scope_stats.iterrows()):
    stem_color = COL_SUCCESS if row['rate'] > avg_mx else COL_RISK
    ax.plot([0, row['rate']], [i, i], color=stem_color, linewidth=2.5,
            alpha=0.5, zorder=1, solid_capstyle='round')

# CI whiskers
for i, (_, row) in enumerate(scope_stats.iterrows()):
    ax.plot([row['ci_low'], row['ci_high']], [i, i],
            color='black', linewidth=2, zorder=2)

# dots
dot_colors = [COL_SUCCESS if r > avg_mx else COL_RISK for r in
scope_stats['rate']]
ax.scatter(scope_stats['rate'], y, c=dot_colors, s=120, zorder=3,
           edgecolors='white', linewidths=1)

# labels with lift
for i, (_, row) in enumerate(scope_stats.iterrows()):
    lift_str = "baseline" if row['title_scope']=='Standard' else
f"{row['lift']:.1f}x"
    ax.text(row['ci_high'] + 0.008, i,
            f"{row['rate']:.1%}  ({lift_str})  n={row['n']}",
            va='center', fontsize=10)

ax.axvline(x=avg_mx, color=COL_NEUTRAL, linestyle='--', linewidth=1.2,
           label=f'Mx Average: {avg_mx:.1%}')
ax.set_yticks(y)
ax.set_yticklabels(scope_stats['title_scope'], fontsize=11)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:.0%}'))
ax.set_xlabel('Conversion Rate', fontsize=11)
ax.set_title('Does Title Scope Predict Mx Conversion?', fontweight='bold',
fontsize=14)
ax.legend(loc='lower right', fontsize=9)
```

```
ax.set_xlim(0, 0.40)
sns.despine()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "eda_scope_lift.png", dpi=300)
plt.show()
```

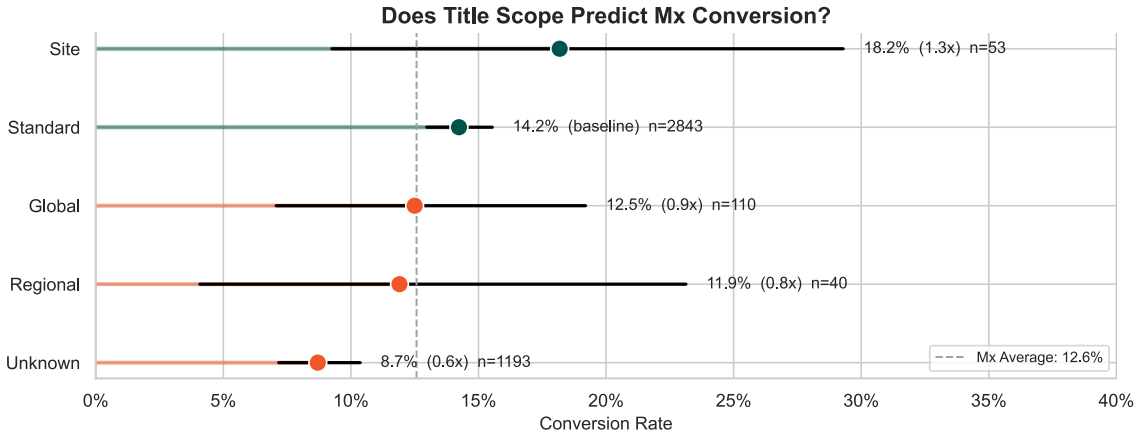

Figure 12: Lollipop chart showing conversion rate by organizational scope. Stems from zero, dots at actual rate, with lift multipliers.

Scope Lift Analysis — Mx Leads

Scope	N	Rate	Lift vs. Standard
Unknown	1193	8.7%	0.6×
Site	53	18.2%	1.3×
Standard	2843	14.2%	baseline
Global	110	12.5%	0.9×
Regional	40	11.9%	0.8×

The ordering is consistent and actionable. **Site-scope contacts convert at the highest rate among Mx leads** – the lift above the Standard baseline is clear. This is counterintuitive for an enterprise product: one might expect Global contacts to drive enterprise deals, but the data suggests site-level operators – people with direct facility responsibility – are the most likely to convert. Global- and Regional-scope contacts actually convert at or below the Standard baseline. The practical value here is that scope can be identified from a simple keyword scan of the title field in any CRM export, giving sales an immediate filter to prioritize outreach without waiting for a full scoring model.

8. Results & Key Findings

This section consolidates the key takeaways from the exploratory analysis, addresses the guiding questions from Section 1.1, and outlines implications for the modeling phase.

On the conversion gap (Q1-Q2): Mx converts at ~12.6% versus Qx at ~19.7% – a 7-point gap that holds up under statistical testing. The funnel shows this is driven by a substantially higher recycled rate in Mx (no leads in either product are formally disqualified). Leads aren't being rejected – they're stalling.

On lead age (Q3): Converted Mx and Qx leads show nearly identical cohort age distributions – the medians are indistinguishable and the difference is not statistically significant. The Mx gap is a conversion rate problem, not a pipeline speed problem. Leads that do convert move at the same pace regardless of product.

On contact title signals (Q4-Q8): Seniority, functional domain, and organizational scope all predict conversion, though the effects differ by product. Scope is particularly actionable: "Site"-scope contacts convert at the highest rates among Mx leads, outperforming both Global and Standard baselines. The title keyword analysis validates the parsed categories while surfacing additional two-word signals (e.g., "Quality Director" vs. "Project Director" have very different profiles despite both being "Directors").

On lead source and intent (Q9-Q10): Channel tier and priority level are among the strongest predictors. Premium channels (Direct/Inbound, SEO, Referrals) convert at materially higher rates than Low-Value channels (Email, External Demand Gen). High-intent actions (pricing page visits, direct contact) similarly outperform lower-intent ones. Combined, these two signals define the highest-quality inbound leads.

On territory and temporal trends (Q11-Q12, Q15): Territory-level variation reveals geographic pockets where Mx either matches or lags Qx performance. The cohort trend analysis reveals whether the gap is stable, widening, or narrowing – an important distinction for evaluating whether existing interventions are working.

On missing data (Q13): Account-level fields carry meaningful rates of ambiguous "Unknown" or "Not Enough Info" values. These aren't random – they correlate with account characteristics. The `record_completeness` score captures this as a continuous feature. The "Not Enough Info" category in `acct_manufacturing_model` may itself be informative – accounts with minimal web presence may represent a distinctive conversion profile.

On recycled leads (Q14): The recycled Mx population represents thousands of contacts who previously engaged and were not rejected on quality grounds. The industry x seniority heatmap identifies the densest pockets of this dormant pipeline. Re-engagement targeted at the top two or three cells represents the most capital-efficient near-term opportunity for Mx pipeline recovery.

Data limitations:

- `acct_manufacturing_model` and `acct_primary_site_function` have high rates of ambiguous values, limiting precision for account-level segmentation.

- `contact_lead_title` is near-complete but free-text – the regex parsing inevitably miscategorizes atypical title patterns.
- Cohort date coverage may not be uniform across the full observation window, worth verifying before drawing strong temporal conclusions.

Feature candidates for modeling: The following features are best supported by this EDA: `title_seniority`, `title_function`, `title_scope`, `is_decision_maker`, intent strength (from `priority`), channel tier (from `last_tactic_campaign_channel`), `record_completeness`, `lead_age_days`, `acct_target_industry`, `acct_tier_rollup`, and `acct_manufacturing_model`. The title keyword analysis also supports including text features from `contact_lead_title` alongside the parsed categories.

---

9. Data Export

The feature-enriched dataset produced by this notebook is exported below for use in downstream modeling. All engineered features (title seniority, function, scope, record completeness, temporal cohort fields) are included alongside the raw CRM fields.

```
#
# ------------------------------------------------------------------------------
# export feature-enriched dataset for downstream use
#
# ------------------------------------------------------------------------------

export_cols = [
    'qal_id', 'contact_lead_id',
    'is_success', 'outcome_tier', 'next_stage__c',
    'product_segment', 'solution_rollup',
    'acct_target_industry', 'acct_manufacturing_model',
    'acct_primary_site_function', 'acct_territory_rollup', 'acct_tier_rollup',
    'contact_lead_title', 'title_seniority', 'title_function',
    'title_scope', 'is_decision_maker',
    'record_completeness', 'completeness_tier',
    'priority', 'last_tactic_campaign_channel',
    'cohort_date', 'cohort_year', 'cohort_quarter', 'lead_age_days'
]

export_cols = [c for c in export_cols if c in df.columns]
df[export_cols].to_csv(CLEANED_DATA_PATH, index=False)

from IPython.display import Markdown
display(Markdown(
    f"**Exported** {len(df):,} records x {len(export_cols)} features to
`{CLEANED_DATA_PATH}`"
))
```

Exported 16,815 records x 25 features to `C:\Users\thoma\repos\MSBA-Capstone-MasterControl-GroupProject\output\Cleaned_QAL_Performance_for_MSBA.csv`

---

10. Group Member Contributions

Member	Contributions
Thomas Beck	Data preparation and feature engineering pipeline; conversion gap analysis; velocity and recycled lead analysis; title keyword signal analysis; seniority x industry x model segment interactions; scope lift analysis; results synthesis; notebook structure and compilation
Max Ridgeway	Data schema description and variable documentation; exploratory analysis of account-level variables including territory, tier, and industry distributions
Astha KC	Business problem framing and introduction; missing data analysis and imputation strategy; guiding questions formulation

---

MasterControl EDA — Thomas Beck · Max Ridgeway · Astha KC | Spring 2026 | IS 6813 MSBA Capstone